

# Introduction to Cryptography

Musard Balliu

Lecture 2

DD2395/DD2391

# Last Lecture

- **Course overview and organization**
- **Introduction to computer security**
- **Security mindset and principles**

**Questions?**

# Today's plan

- **Introduction to cryptography**
- **Security properties of interest**
- **Symmetric key cryptography**
  - **One Time Pad, Block ciphers**

# What is cryptography?

# What is cryptography?

- Older definition: The study of secure communication over insecure channels
- Newer definition: Provide rigorous guarantees about the data and computation in the presence of an attacker
  - Not just *confidentiality* but also *integrity* and *authenticity* (we'll get a taste of it in these lectures)
- Modern cryptography involves a lot of math (we won't get into many details in this course)

# Don't try this at home!

- We will overview the basic building blocks of cryptography, but you should NEVER try to write your own cryptographic algorithms
- It's very easy to make a mistake that makes your code insecure
  - Required formal rigor and tricky edge cases that we won't cover
  - One small bug could compromise the security of your code
- Instead, use existing well-vetted cryptographic libraries
  - This lecture is as much about making you a good *consumer* of cryptography

# Definitions

# Meet Alice, Bob, Eve, and Mallory

- Alice and Bob: The main characters trying to send messages to each other over an insecure communication channel
- Eve: An **eavesdropper** who can read any data sent over the channel
  - Also called an honest-but-curious attacker
- Mallory: A **manipulator** who can read and modify any data sent over the channel
  - Also called a malicious attacker

# Meet Alice, Bob, Eve, and Mallory

- We often describe cryptographic problems using a common cast of characters
- One scenario:
  - Alice wants to send a message to Bob.
  - However, Eve is going to *eavesdrop* on the communication channel.
  - How does Alice send the message to Bob without Eve learning about the message?
- Another scenario:
  - Bob wants to send a message to Alice.
  - However, Mallory is going to *tamper* with the communication channel.
  - How does Bob send the message to Alice without Mallory changing the message?

# Three Goals of Cryptography

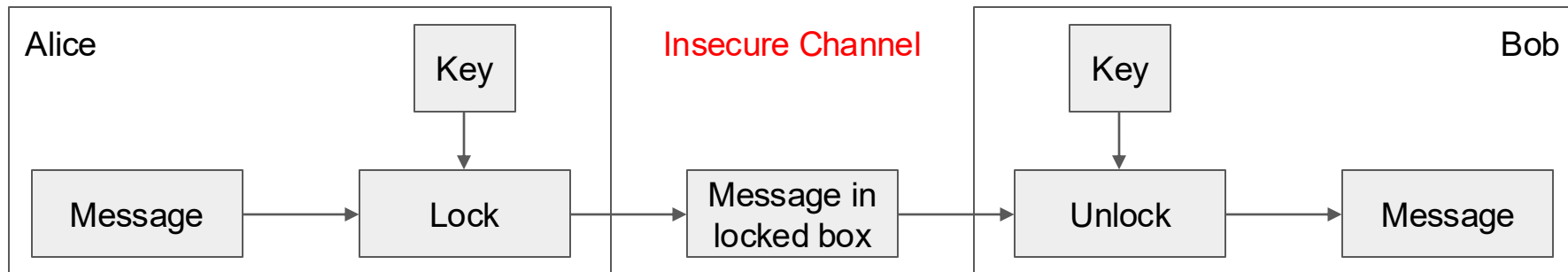
- In cryptography, there are three common properties that we want on our data
- **Confidentiality**: An adversary cannot *read* our messages.
- **Integrity**: An adversary cannot *change* our messages without being detected.
- **Authenticity**: I can prove that this message came from the person who claims to have written it.
  - Integrity and authenticity are closely related properties...
    - Before I can prove that a message came from a certain person, I have to prove that the message wasn't changed!

# Keys

- The most basic building block of any cryptographic scheme: The **key**
- Properly chosen and guarded keys “power” the security of our cryptographic algorithms
- Assumption: The attacker knows all the algorithms we use – cf. Principle of open design. The only information the attacker is missing is the secret key(s).
- Two models of keys:
  - **Symmetric key model:** Alice and Bob both know the value of the same secret key.
  - **Asymmetric key model:** A user has two keys, a secret key and a public key.

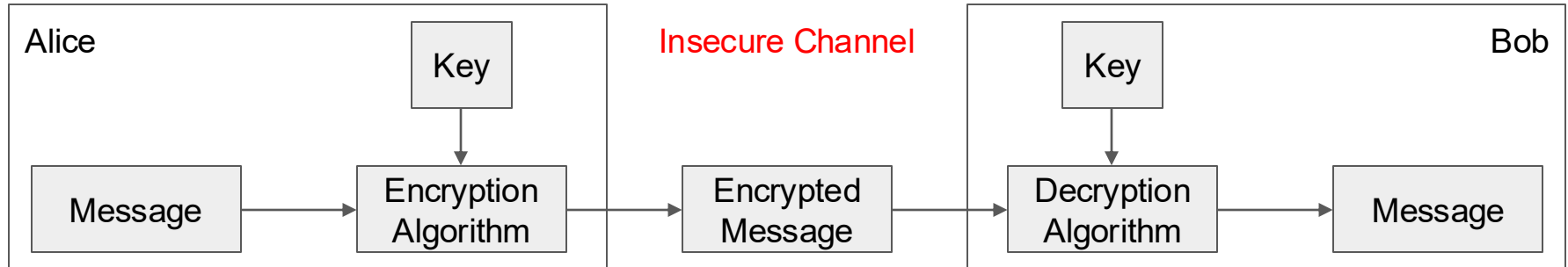
# Confidentiality

- **Confidentiality:** An adversary cannot *read* our messages.
- **Analogy:** Locking and unlocking the message
  - Alice uses the key to lock the message in a box
  - Alice sends the message (locked in the box) over the insecure channel
  - Eve sees the locked box, but cannot access the message without the key
  - Bob receives the message (locked in the box) and uses the key to unlock the message



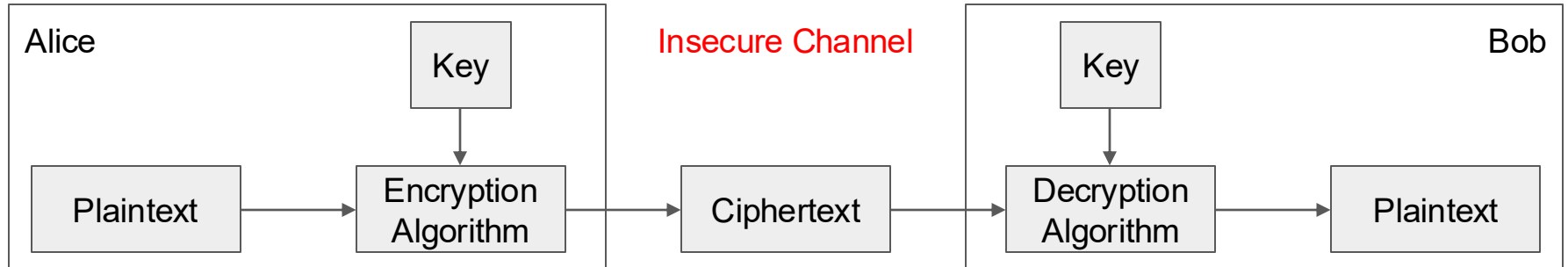
# Confidentiality

- Schemes provide confidentiality by **encrypting** messages
  - Alice uses the key to **encrypt** the message: Change the message into a scrambled form
  - Alice sends the encrypted message over the insecure channel
  - Eve sees the encrypted message, but cannot figure out the original message without the key
  - Bob receives the encrypted message and uses the key to **decrypt** the message back into its original form



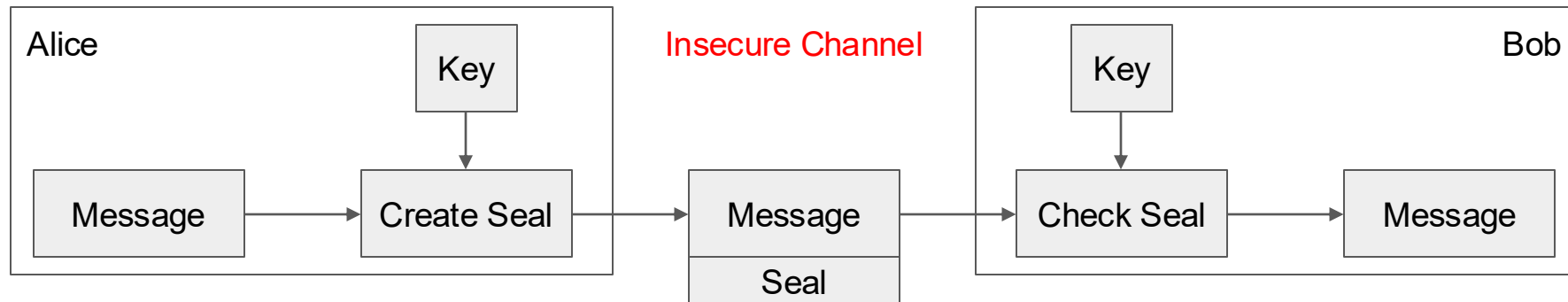
# Confidentiality

- **Plaintext:** The original message
- **Ciphertext:** The encrypted message



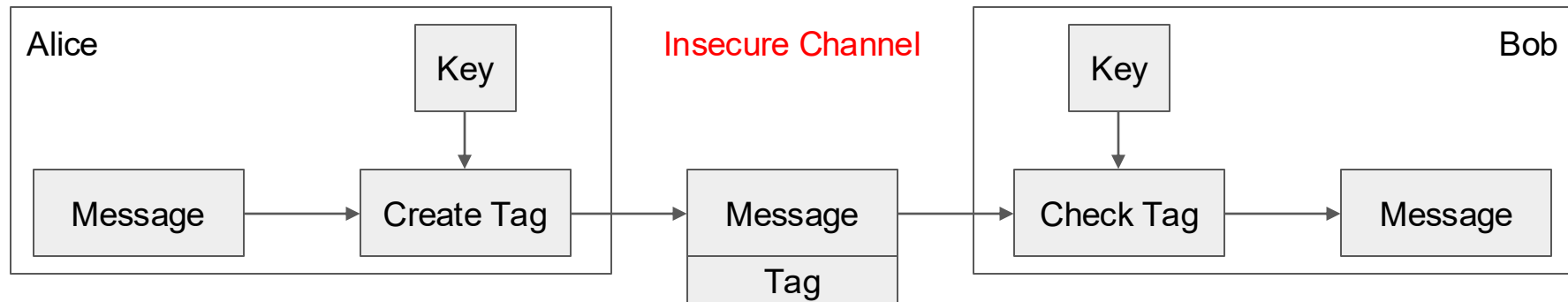
# Integrity (and Authenticity)

- **Integrity:** An adversary cannot *change* our messages without being detected.
- **Analogy:** Adding a seal on the message
  - Alice uses the key to add a special seal on the message (e.g. puts tape on the envelope)
  - Alice sends the message and the seal over the insecure channel
  - If Mallory tampers with the message, she'll break the seal (e.g. break the tape on the envelope)
  - Without the key, Mallory cannot create her own seal
  - Bob receives the message and the seal and checks that the seal has not been broken



# Integrity (and Authenticity)

- Schemes provide integrity by adding a **tag** or **signature** on messages
  - Alice uses the key to generate a special tag for the message
  - Alice sends the message and the tag over the insecure channel
  - If Mallory tampers with the message, the tag will no longer be valid
  - Bob receives the message and the tag and checks that the tag is still valid



# Threat Models

- What if Eve can do more than eavesdrop?
- Real-world schemes are often vulnerable to more sophisticated attackers, so cryptographers have created more sophisticated threat models too
- Some threat models for analyzing confidentiality:

|                                    | Can Eve trick Alice into encrypting messages of Eve's choosing? | Can Eve trick Bob into decrypting messages of Eve's choosing? |
|------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------|
| <b>Ciphertext-only</b>             | No                                                              | No                                                            |
| <b>Chosen-plaintext</b>            | Yes                                                             | No                                                            |
| <b>Chosen-ciphertext</b>           | No                                                              | Yes                                                           |
| <b>Chosen plaintext-ciphertext</b> | Yes                                                             | Yes                                                           |

# Threat Models

- In this lectures, we'll focus on the chosen plaintext attack model
- In practice, cryptographers use the chosen plaintext-ciphertext model
  - It's the most powerful
  - It can actually be defended against

|                                    | Can Eve trick Alice into encrypting messages of Eve's choosing? | Can Eve trick Bob into decrypting messages of Eve's choosing? |
|------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------|
| <b>Ciphertext-only</b>             | No                                                              | No                                                            |
| <b>Chosen-plaintext</b>            | Yes                                                             | No                                                            |
| <b>Chosen-ciphertext</b>           | No                                                              | Yes                                                           |
| <b>Chosen plaintext-ciphertext</b> | Yes                                                             | Yes                                                           |

# Cryptography Roadmap

|                                      | <b>Symmetric-key</b>                                                                               | <b>Asymmetric-key</b>                                                                         |
|--------------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Confidentiality</b>               | <ul style="list-style-type: none"><li>• One-time pads</li><li>• Block ciphers (e.g. AES)</li></ul> | <ul style="list-style-type: none"><li>• RSA encryption</li><li>• ElGamal encryption</li></ul> |
| <b>Integrity,<br/>Authentication</b> | <ul style="list-style-type: none"><li>• Hash functions</li><li>• MACs (e.g. HMAC)</li></ul>        | <ul style="list-style-type: none"><li>• Digital signatures (e.g. RSA signatures)</li></ul>    |

# Symmetric-Key Encryption

# Cryptography Roadmap

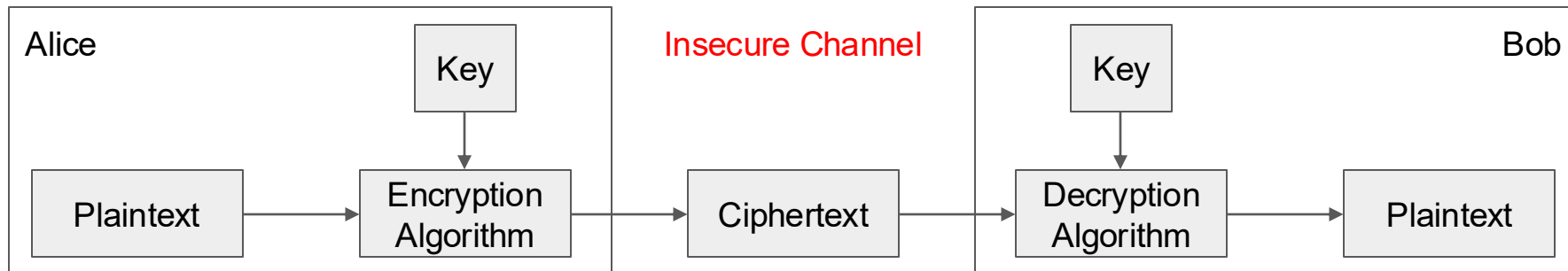
|                                      | <b>Symmetric-key</b>                                                                               | <b>Asymmetric-key</b>                                                                         |
|--------------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Confidentiality</b>               | <ul style="list-style-type: none"><li>• One-time pads</li><li>• Block ciphers (e.g. AES)</li></ul> | <ul style="list-style-type: none"><li>• RSA encryption</li><li>• ElGamal encryption</li></ul> |
| <b>Integrity,<br/>Authentication</b> | <ul style="list-style-type: none"><li>• Hash functions</li><li>• MACs (e.g. HMAC)</li></ul>        | <ul style="list-style-type: none"><li>• Digital signatures (e.g. RSA signatures)</li></ul>    |

# Symmetric-Key Encryption

- The next few schemes are symmetric-key encryption schemes
  - **Encryption schemes** aim to provide *confidentiality* (but not integrity or authentication)
  - **Symmetric-key** means Alice and Bob share the same secret key that the attacker doesn't know
    - Don't worry about how Alice and Bob share the key for now
- For modern schemes, we're going to assume that messages are *bitstrings*
  - **Bitstring**: A sequence of bits (0 or 1), e.g. 11010101001001010
  - Text, images, etc. can be converted into bitstrings before encryption, so bitstrings are a useful abstraction. After all, everything in a computer is just a sequence of bits!

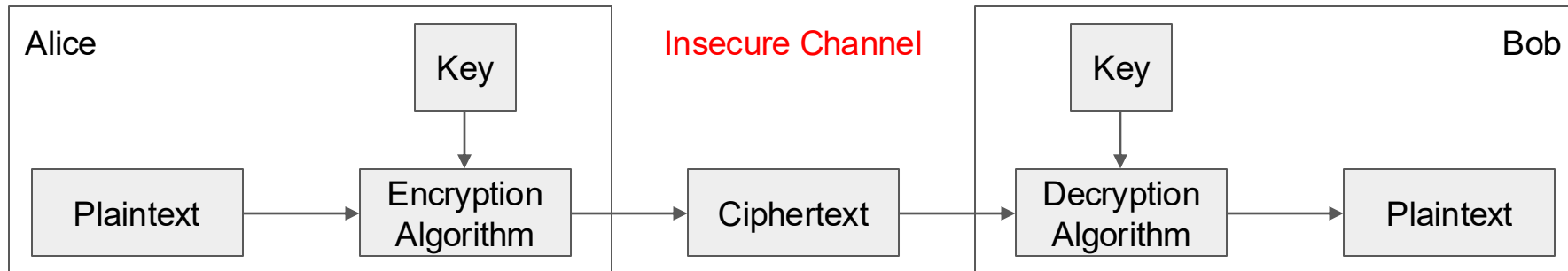
# Symmetric-Key Encryption: Definition

- A symmetric-key encryption scheme has three algorithms:
  - $\text{KeyGen}() \rightarrow K$ : Generate a key  $K$
  - $\text{Enc}(K, M) \rightarrow C$ : Encrypt a **plaintext**  $M$  using the key  $K$  to produce **ciphertext**  $C$
  - $\text{Dec}(K, C) \rightarrow M$ : Decrypt a ciphertext  $C$  using the key  $K$



# Symmetric-Key Encryption: Definition

- What properties do we want from a symmetric encryption scheme?
  - **Correctness:** Decrypting a ciphertext should result in the message that was originally encrypted
    - $\text{Dec}(K, \text{Enc}(K, M)) = M$  for all  $K \leftarrow \text{KeyGen}()$  and  $M$
  - **Efficiency:** Encryption/decryption algorithms should be fast on a standard computer
  - **Security:** Confidentiality



# Defining Confidentiality

- Recall our definition of confidentiality from earlier: “An adversary cannot read our messages”
  - This definition isn’t very specific
    - What if Eve can read the first half of Alice’s message, but not the second half?
    - What if Eve figures out that Alice’s message starts with “Dear Bob”?
  - This definition doesn’t account for prior knowledge
    - What if Eve already knew that Alice’s message ends in “Sincerely, Alice”?
    - What if Eve knows that Alice’s message is “BUY!” or “SELL” but doesn’t know which?

Q: How would you define confidentiality?

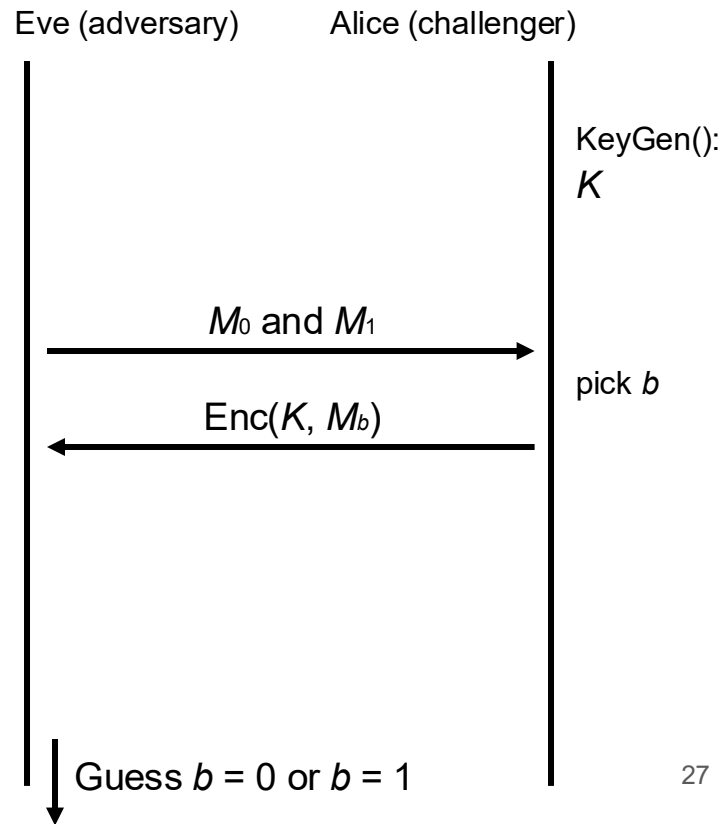
# Defining Confidentiality

- A better definition of confidentiality: The ciphertext should not give the attacker *any additional information* about the plaintext.
- Let's design an experiment/security game to test our definition

# Security game: first attempt at confidentiality

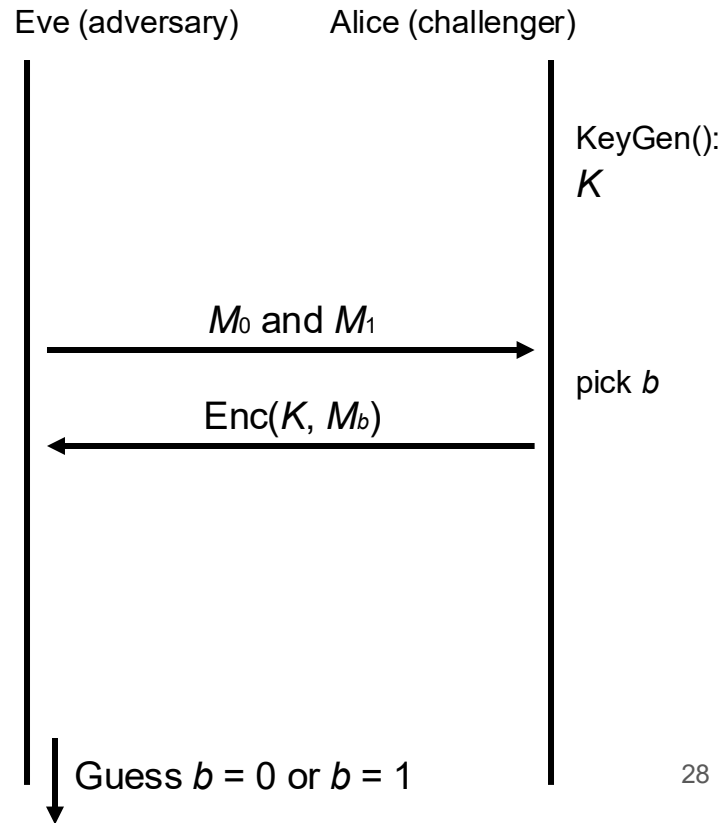
1. Eve issues a pair of plaintexts  $M_0$  and  $M_1$  to Alice of the same length
2. Alice randomly chooses either  $M_0$  or  $M_1$  to encrypt and sends the encryption back
  - a. Alice does not tell Eve which one was encrypted!
3. Eventually, Eve outputs a guess as to whether Alice encrypted  $M_0$  or  $M_1$

Q: If the scheme provides confidentiality, what chance does the attacker have to guess  $b$ ?



# Security game: intuition

- If the scheme is confidential, Eve can only guess with probability  $1/2$ , which is no different than if Eve hadn't seen the ciphertext at all
- In other words: the ciphertext gave Eve no *additional* information about which plaintext was sent!

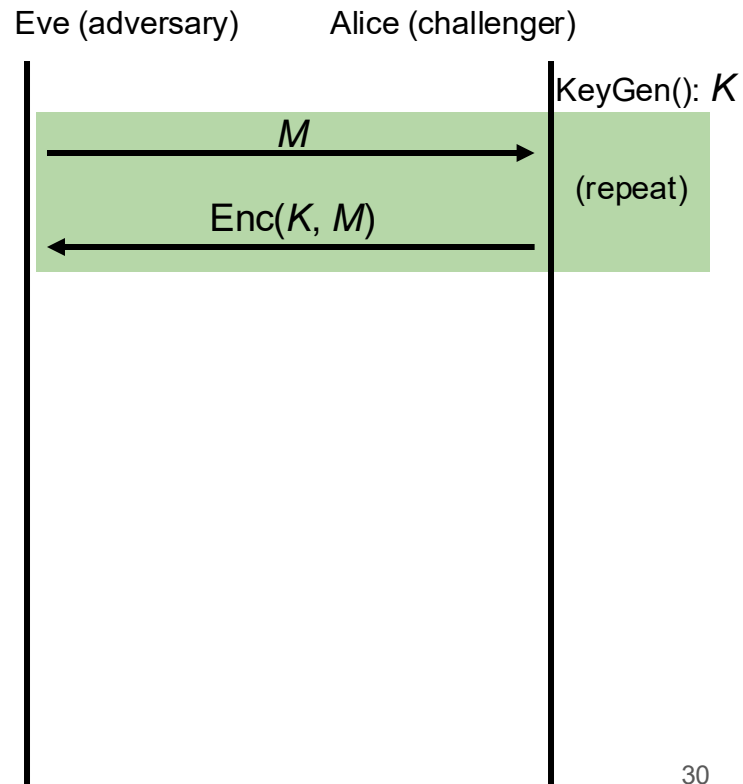


# Defining Confidentiality: IND-CPA

- Recall our threat model: Eve can also perform a **chosen plaintext attack**
  - Eve can trick Alice into encrypting arbitrary messages of Eve's choice
  - We can adapt our experiment to account for this threat model
- A better definition of confidentiality: Even if Eve is able to trick Alice into encrypting messages, Eve can still only guess what message Alice sent with probability  $1/2$ .
  - This definition is called **IND-CPA** (indistinguishability under chosen plaintext attack)
- Cryptographic properties are often defined in terms of “games” that an adversary can either “win” or “lose”
  - We will use one to define confidentiality precisely

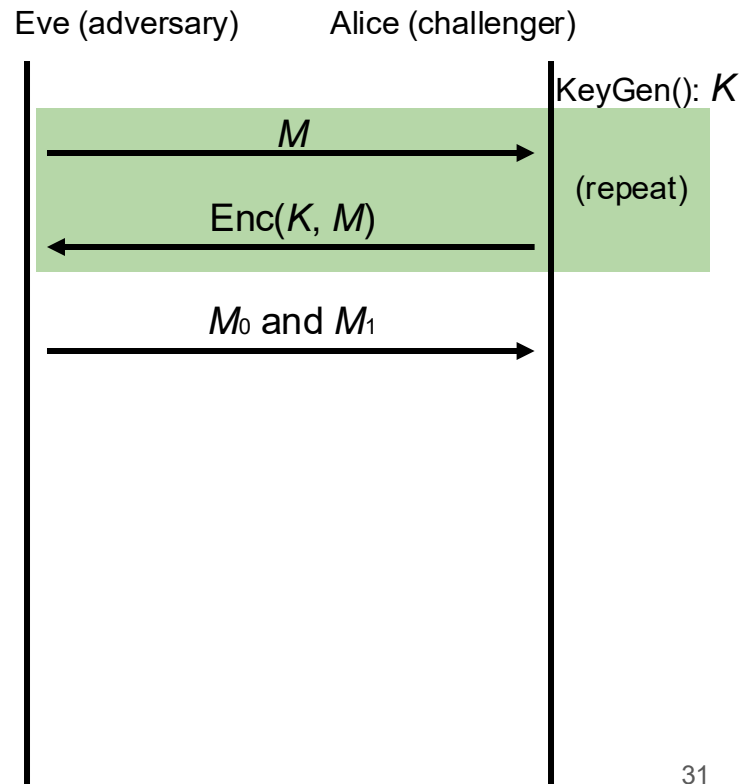
# Defining Confidentiality: IND-CPA

1. Eve may choose plaintexts to send to Alice and receives their ciphertexts



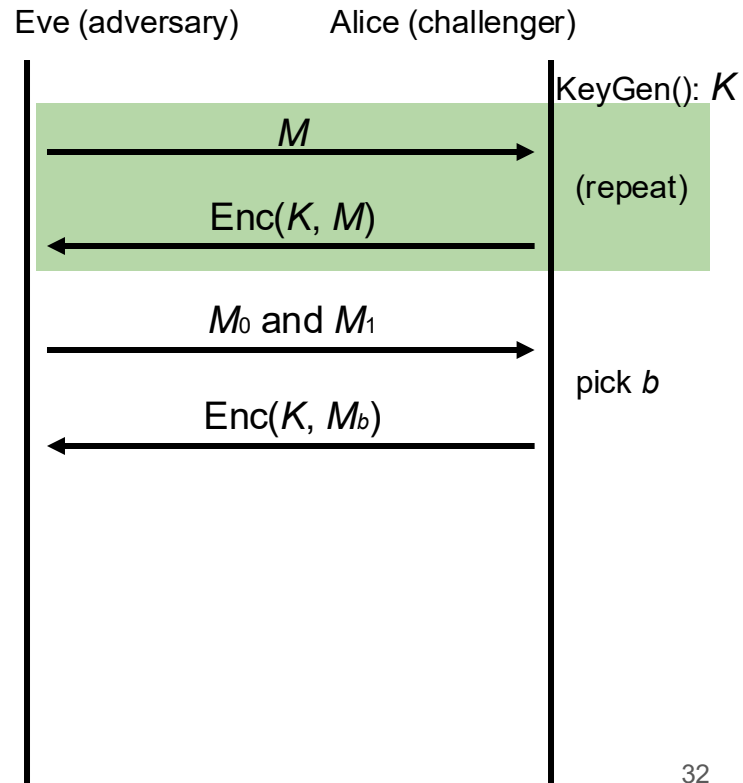
# Defining Confidentiality: IND-CPA

1. Eve may choose plaintexts to send to Alice and receives their ciphertexts
2. Eve issues a pair of plaintexts  $M_0$  and  $M_1$  to Alice



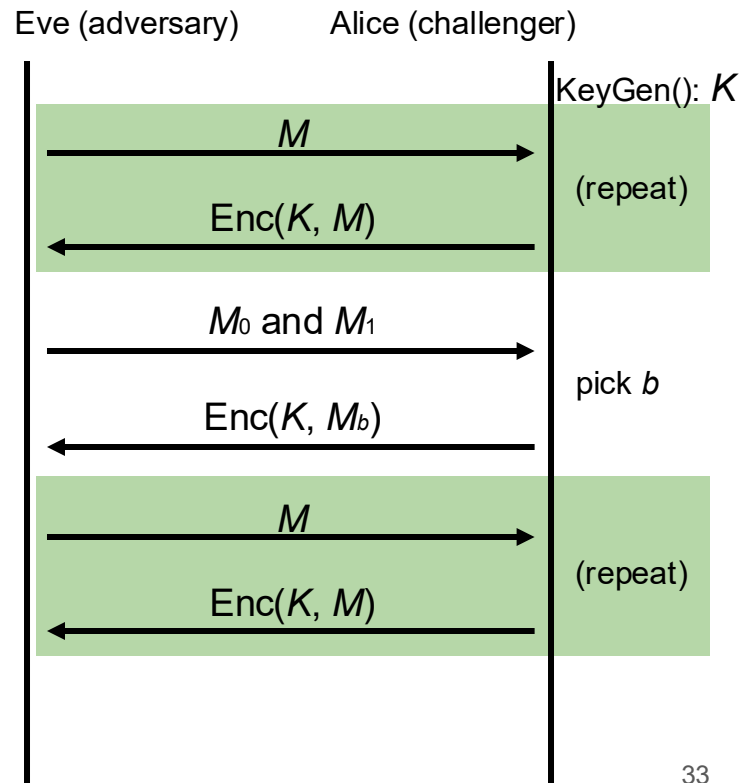
# Defining Confidentiality: IND-CPA

1. Eve may choose plaintexts to send to Alice and receives their ciphertexts
2. Eve issues a pair of plaintexts  $M_0$  and  $M_1$  to Alice
3. Alice randomly chooses either  $M_0$  or  $M_1$  to encrypt and sends the encryption back
  - a. Alice does not tell Eve which one was encrypted!



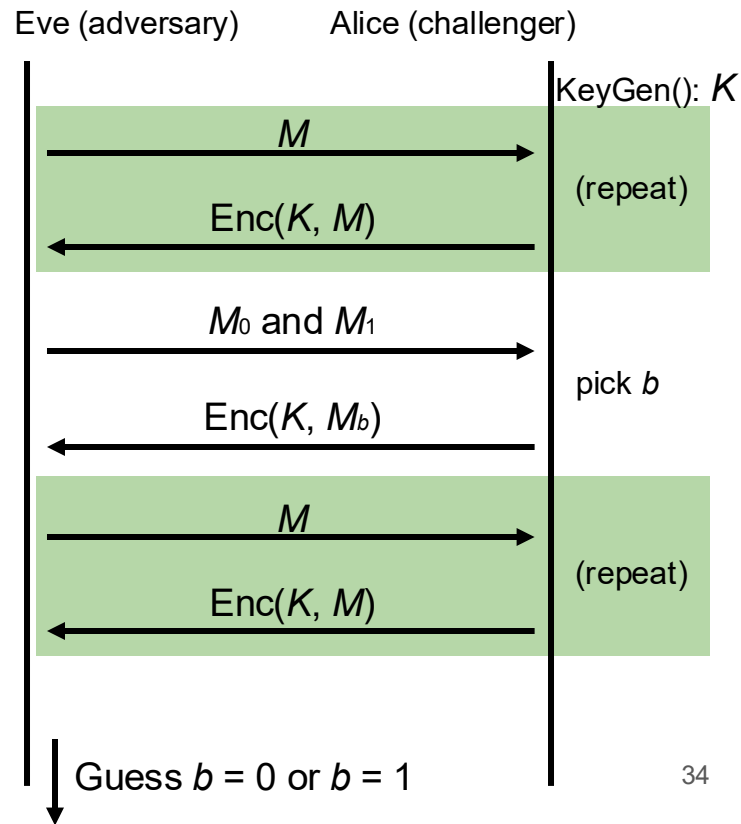
# Defining Confidentiality: IND-CPA

1. Eve may choose plaintexts to send to Alice and receives their ciphertexts
2. Eve issues a pair of plaintexts  $M_0$  and  $M_1$  to Alice
3. Alice randomly chooses either  $M_0$  or  $M_1$  to encrypt and sends the encryption back
  - a. Alice does not tell Eve which one was encrypted!
4. Eve may again choose plaintexts to send to Alice and receives their ciphertexts



# Defining Confidentiality: IND-CPA

1. Eve may choose plaintexts to send to Alice and receives their ciphertexts
2. Eve issues a pair of plaintexts  $M_0$  and  $M_1$  to Alice
3. Alice randomly chooses either  $M_0$  or  $M_1$  to encrypt and sends the encryption back
  - a. Alice does not tell Eve which one was encrypted!
4. Eve may again choose plaintexts to send to Alice and receives their ciphertexts
5. Eventually, Eve outputs a guess as to whether Alice encrypted  $M_0$  or  $M_1$



# Defining Confidentiality: IND-CPA

- If Eve correctly guesses which message Alice encrypted, then Eve wins. Otherwise, she loses.
- How does Eve guess whether  $M_0$  or  $M_1$  was encrypted? What strategy does she use?
  - We don't *assume* she uses a particular strategy; Eve represents all possible strategies
- Proving insecurity: There exists at least *one* strategy that can win the IND-CPA game with probability  $> 1/2$ 
  - $1/2$  is the probability of winning by random guessing
  - If you can be better than random, then the ciphertext has leaked information, and Eve is able to learn it and use it to gain an advantage!
- Proving security: For *all* (polynomial-time) attackers/Eve-s, the probability of winning the IND-CPA game is at most  $1/2$

# Edge Cases: Length

- Cryptographic schemes are (usually) allowed to leak the length of the message
  - To hide length: All messages must always be the same length
    - 16-byte messages: We can't encrypt large messages (images, videos, etc.)!
    - 1-GB messages: Sending small messages (text, Tweets, etc.) needs 1 GB of bandwidth!
    - This is impractical
  - Applications can choose to hide length by *padding* their own messages to the maximum possible length before encrypting
- In the IND-CPA game:  $M_0$  and  $M_1$  must be the same length
  - To break IND-CPA, Eve must learn something other than message length

# Edge Cases: Attacker Runtime

- Some schemes are theoretically vulnerable, but secure in any real-world setting
  - If an attack takes longer than the life of the solar system to complete, it probably won't happen!
- In the IND-CPA game: Eve is limited to a practical runtime
  - One common practical limit: Eve is limited to polynomial runtime algorithms (no exponential-time algorithms)

# Edge Cases: Negligible Advantage

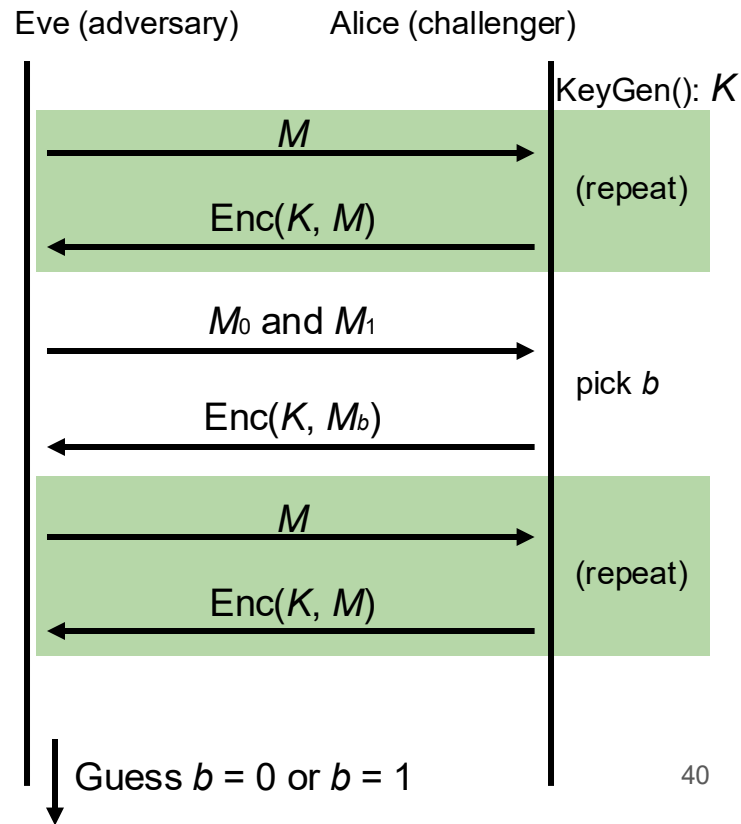
- Sometimes it's possible for Eve to win with probability  $1/2 + 1/2^{128}$ 
  - This probability is greater than  $1/2$ , but it's so close to  $1/2$  that it's as good as  $1/2$ .
  - Eve's advantage is so small that she can't use it for any practical attacks
- In the IND-CPA game: The scheme is secure even if Eve can win with probability  $\leq 1/2 + \epsilon$ , where  $\epsilon$  is *negligible*
  - The actual mathematical definition of negligible is out of scope
  - Example:  $1/2 + 1/2^{128}$ : Negligible advantage
  - Example:  $2/3$ : Non-negligible advantage

# Edge Cases: Negligible Advantage

- Defining negligibility mathematically:
  - Advantage of the adversary should be exponentially small, based on the security parameters of the algorithm
  - Example: For an encryption scheme with a  $k$ -bit key, the advantage should be  $O(1/2^k)$
- Defining negligibility practically:
  - A  $1/2^{128}$  probability is currently unlikely
  - A  $1/2^{20}$  probability is fairly likely
  - In between these extremes, it can be messy
    - Different algorithms run faster or slower and have their own security parameters
    - Computers get more powerful over time
    - Recall: Know your threat model!
- **Takeaway:** For now,  $2^{80}$  is a reasonable threshold, but this will change over time!

# IND-CPA: Putting it together

1. Eve may choose plaintexts to send to Alice and receives their ciphertexts
  2. Eve issues a pair of plaintexts  $M_0$  and  $M_1$  to Alice
  3. Alice randomly chooses either  $M_0$  or  $M_1$  to encrypt and sends the encryption back
    - Alice does not tell Eve which one was encrypted!
  4. Eve may again choose plaintexts to send to Alice and receives their ciphertexts
  5. Eventually, Eve outputs a guess as to whether Alice encrypted  $M_0$  or  $M_1$
- An encryption scheme is IND-CPA secure if for all polynomial time attackers Eve:
    - Eve can win with probability  $\leq 1/2 + \epsilon$ , where  $\epsilon$  is *negligible*.



# One-Time Pads

# Cryptography Roadmap

|                              | Symmetric-key                                                                                      | Asymmetric-key                                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Confidentiality              | <ul style="list-style-type: none"><li>● One-time pads</li><li>● Block ciphers (e.g. AES)</li></ul> | <ul style="list-style-type: none"><li>● RSA encryption</li><li>● ElGamal encryption</li></ul> |
| Integrity,<br>Authentication | <ul style="list-style-type: none"><li>● Hash functions</li><li>● MACs (e.g. HMAC)</li></ul>        | <ul style="list-style-type: none"><li>● Digital signatures (e.g. RSA signatures)</li></ul>    |

# Review: XOR

The XOR operator takes two bits and outputs one bit:

|                  |
|------------------|
| $0 \oplus 0 = 0$ |
| $0 \oplus 1 = 1$ |
| $1 \oplus 0 = 1$ |
| $1 \oplus 1 = 0$ |

Useful properties of XOR:

|                                                 |
|-------------------------------------------------|
| $x \oplus 0 = x$                                |
| $x \oplus x = 0$                                |
| $x \oplus y = y \oplus x$                       |
| $(x \oplus y) \oplus z = x \oplus (y \oplus z)$ |
| $(x \oplus y) \oplus x = y$                     |

# Review: XOR Algebra

- Algebra works on XOR too

|                                    |                          |
|------------------------------------|--------------------------|
| $y \oplus 1 = 0$                   | Goal: Solve for y        |
| $y \oplus 1 \oplus 1 = 0 \oplus 1$ | XOR both sides by 1      |
| $y = 1$                            | Simplify with identities |

# One-Time Pads: Key Generation

Alice

$K$

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|

The key  $K$  is a randomly-chosen bitstring.

Recall: We are in the symmetric-key setting, so we'll assume Alice and Bob both know this key.

# One-Time Pads: Encryption

Alice

$K$

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|

$M$

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|

The plaintext  $M$  is the bitstring that Alice wants to encrypt.

Idea: Use XOR to scramble up  $M$  with the bits of  $K$ .

# One-Time Pads: Encryption

Alice

|     |          |          |          |          |          |          |          |          |          |          |          |          |
|-----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| $K$ | 0        | 1        | 1        | 0        | 0        | 1        | 0        | 1        | 0        | 1        | 1        | 1        |
|     | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ |
| $M$ | 1        | 0        | 0        | 1        | 1        | 0        | 0        | 1        | 0        | 1        | 0        | 0        |
|     | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        |
| $C$ | 1        | 1        | 1        | 1        | 1        | 1        | 0        | 0        | 0        | 0        | 1        | 1        |

Encryption algorithm: XOR each bit of  $K$  with the matching bit in  $M$ .

The ciphertext  $C$  is the encrypted bitstring that Alice sends to Bob over the insecure channel.

# One-Time Pads: Decryption

Bob

|     |   |   |   |   |   |   |   |   |   |   |   |   |
|-----|---|---|---|---|---|---|---|---|---|---|---|---|
| $K$ | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 |
| $C$ | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |

Bob receives the ciphertext  $C$ . Bob knows the key  $K$ . How does Bob recover  $M$ ?

# One-Time Pads: Decryption

Bob

|     |          |          |          |          |          |          |          |          |          |          |          |          |
|-----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| $K$ | 0        | 1        | 1        | 0        | 0        | 1        | 0        | 1        | 0        | 1        | 1        | 1        |
|     | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ | $\oplus$ |
| $C$ | 1        | 1        | 1        | 1        | 1        | 1        | 0        | 0        | 0        | 0        | 1        | 1        |
|     | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        | ↓        |
| $M$ | 1        | 0        | 0        | 1        | 1        | 0        | 0        | 1        | 0        | 1        | 0        | 0        |

Decryption algorithm: XOR each bit of  $K$  with the matching bit in  $C$ .

# One-Time Pad

- KeyGen()
  - Randomly generate an  $n$ -bit key, where  $n$  is the length of your message
    - Recall: We assume that Alice and Bob can securely share this key
    - For one-time pads, we generate a *new key* for every message
- $\text{Enc}(K, M) = K \oplus M$ 
  - Bitwise XOR  $M$  and  $K$  to produce  $C$ 
    - In other words: XOR the  $i$ th bit of the plaintext with the  $i$ th bit of the key.
    - $C_i = K_i \oplus M_i$
  - Alice and Bob use a different key for each encryption (this is the “one-time” in one-time pad).
- $\text{Dec}(K, C) = K \oplus C$ 
  - Bitwise XOR  $C$  and  $K$  to produce  $M$ 
    - $M_i = K_i \oplus C_i$

# One-Time Pad: Correctness

- Correctness: If we encrypt and then decrypt, we should get the original message back

$$\text{Enc}(K, M) = K \oplus M$$

Definition of encryption

$$\text{Dec}(K, \text{Enc}(K, M)) = \text{Dec}(K, K \oplus M)$$

Decrypting the ciphertext

$$= K \oplus (K \oplus M)$$

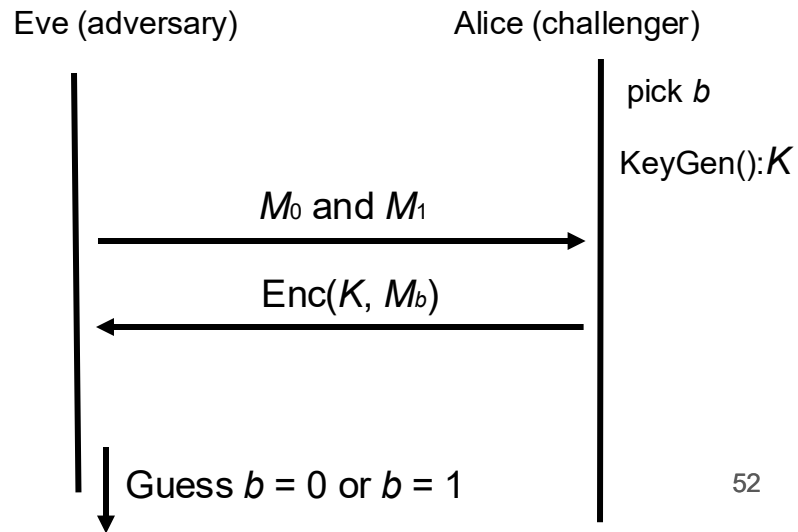
Definition of decryption

$$= M$$

XOR property

# One-Time Pad: Security

- Recall our definition of confidentiality: The ciphertext should not give the attacker any additional information about the plaintext
- Recall our preliminary experiment (before IND-CPA) to test confidentiality from earlier:
  - If the probability that Eve correctly guesses which message was sent is  $1/2$ , then the encryption scheme is confidential



# One-Time Pad: Security

Possibility 0: Alice sends  $\text{Enc}(K, M_0)$

The ciphertext is  $C = K \oplus M_0$

Therefore,  $K = C \oplus M_0$

Possibility 1: Alice sends  $\text{Enc}(K, M_1)$

The ciphertext is  $C = K \oplus M_1$

Therefore,  $K = C \oplus M_1$

$K$  was chosen randomly, so both possibilities are equally possible

Eve has learned no new information, so the scheme is *perfectly secure*

By “perfectly” we mean that any attacker has chance of winning the security game exactly  $\frac{1}{2}$  (not  $\frac{1}{2} + \epsilon$ )

# Is One-Time Pad IND-CPA secure?

1. Eve may choose plaintexts to send to Alice and receives their ciphertexts
2. Eve issues a pair of plaintexts  $M_0$  and  $M_1$  to Alice
3. Alice randomly chooses either  $M_0$  or  $M_1$  to encrypt and sends the encryption back
  - a. Alice does not tell Eve which one was encrypted!
4. Eve may again choose plaintexts to send to Alice and receives their ciphertexts
5. Eventually, Eve outputs a guess as to whether Alice encrypted  $M_0$  or  $M_1$

An encryption scheme is IND-CPA secure if for all polynomial time attackers Eve: Eve can win with probability  $\leq 1/2 + \epsilon$ , where  $\epsilon$  is *negligible*.

A: No, because it requires that the scheme is able to reuse a key  $K$  without helping the attacker distinguish. In repeat phase, attacker can ask for  $M_0$

